Observability, Debugging & CI/CD

Execution → Prediction → Meaning

Bryan Bischof

22 Apr 2026

Today's journey

Units 9–12 each answered: *how do I prove this works before shipping?*

This unit

How do you keep proving it once the system is **in production** — and how do you debug it when it stops working?

The steel thread

Execution → Prediction → Meaning

Observability across three eras: software, ML, LLM. One discipline; the hidden state expands.

Framing (first half)

- The steel thread
- MLOps lineage (Sculley + CACE)
- Three debugging paradigms
- Evals as observability (vendor debate)

Five key ideas (second half)

1. Trace is the unit of debugging
2. Root cause vs. symptom
3. CI as the eval harness
4. Online monitoring & drift
5. Alerting on quality, not latency

Today's session — same lens all day

Trace-first debugging, structured logging across agent and MCP boundaries, root cause vs. symptom discipline, golden datasets as CI tests, regression gates on prompt and model changes, online drift detection, and alerting on eval degradation (not just latency).

The continuous loop from Unit 3

Analyze → Measure → Improve
now running on the production clock.

The artifacts from eval (golden datasets, rubrics, traces, invariants) are the *same* artifacts you run in CI, emit from production, and alert on when they regress.

After Units 9, 10, 11, 12 — what we reuse

Earlier-unit idea	How it shows up in ops
Trace / trajectory (U9)	Primary observability primitive: agent turn, pipeline record, tool call — each emits a structured trace.
Harness (U9)	Runtime that emits traces, enforces checkpoints, fires alerts when an invariant fails.
Multi-agent attribution (U10)	Span IDs + parent links let you answer *which* agent regressed, not just "the system is worse."
Three-layer eval (U11)	Invariants + sampling + judge at offline time is the same recipe as online; only cadence and budget change.
Extract-then-validate (U11)	Same pattern powers deploy-time gates: generate → validate → block promotion.
Analyze–Measure–Improve (U3)	Inner loop of observability; now runs weekly instead of once before ship.
Model adaptation (U12)	Every fine-tune / adapter swap / base-model bump is a **regression event** that must pass the same CI gates as a prompt change.
Determinism × agency (U9)	Tells you *what* to alert on: deterministic steps fail loud; agentic steps fail quiet.

Outline

What "observability" actually means

Definition

Observability is the craft of **recovering what happened inside a running system from the signals it emits about itself.**

You rarely have a live production system under a debugger. You have events, measurements, causal chains — *and your job is to reconstruct behavior from those signals.*

Where the word comes from:

- **Control theory** (Kalman, 1960): a system is *observable* if its internal state can be inferred from its outputs alone.
- **Distributed systems** (Sridharan, Majors, Honeycomb — mid-2010s): re-imported as a deliberate contrast with pre-wired monitoring.

Three canonical signals (that pre-date LLMs)

- **Traces:** the causal chain of work as a request moves through a system; structured as parent / child spans so you can see which step called which.
- **Metrics:** numerical aggregates over time — request rate, error rate, latency percentiles — cheap to store, cheap to alert on.
- **Logs:** structured events emitted from the components themselves; usually JSON today, not free-form text.

These three haven't gone away. They reappear throughout this unit.

What *does* change across eras is **which hidden state matters.**

The arc: *which* hidden state matters?

Software observability
Did the code run?

- Which service handled it?
- Where did latency accumulate?
- Did a dependency fail?

$\Rightarrow$ execution state

MLOps
Does the model still fit reality?

- Drift / skew
- Degraded prediction
- Stale lineage

$\Rightarrow$ + model / data state

LLMOps
Did we produce the right meaning, grounded?

- Hallucination, bad grounding
- Wrong tool use
- Unsafe or costly chains

$\Rightarrow$ + semantic / context state

The **locus of correctness rises up the stack.** Infrastructure telemetry stays necessary; it just stops being sufficient.

“But stochastic systems are new!”

No, they are not.

Data science and ML practitioners have been shipping and operating probabilistic systems in production for at least a decade before “AI engineer” was a job title.

A great deal of what the software-engineering side is currently rediscovering about LLMs already has answers in the MLOps literature.

Consequence for this course

The genuinely new parts are fewer than the hype suggests. Most of this unit is **old ideas applied to a new trace shape** — and that is a feature, not a bug.

Follow one request through three eras

A. Software era. A request arrives; you trace ingress → services → DB → response.
Where did this request fail or slow down?

B. MLOps era. Same request hits a classifier. You now *also* trace feature generation, model version, evaluation history, drift / skew, eventual ground truth.
Did the request execute correctly **and** *is the model still valid for the world it operates in?*

C. LLMOps era. Same request reaches an agent. You also trace the prompt template, retrieval query, every retrieved document and its provenance, every model call, every tool invocation, reasoning steps, safety checks, tokens, eval feedback.
Did the system produce a useful, safe, grounded answer — and can I explain why?

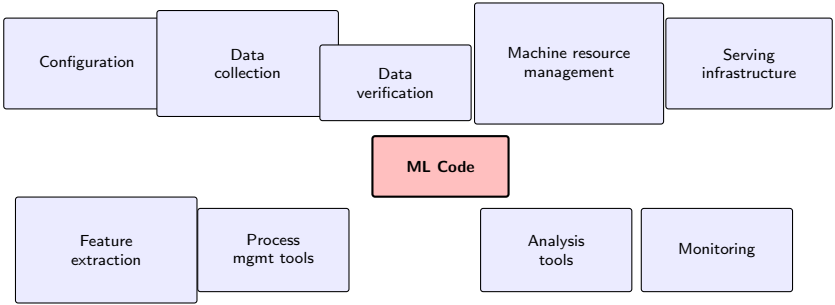
Same trace shape, wider and deeper

The infrastructure trace is still there. It just has strictly more nested children.

Outline

Sculley et al., 2015: the iceberg

Hidden Technical Debt in Machine Learning Systems (NeurIPS 2015) — the "high-interest credit card" paper.



Failure modes the paper named:
CACE ("Changing Anything Changes Everything").
Undeclared consumers. Hidden feedback loops.
Unstable data dependencies. Glue code.
Pipeline jungles. Configuration debt.

The operational claim:
Testing data invariants and monitoring prediction quality are **operational**, not research, concerns.

A good fraction of MLOps since is a debt payment against something this paper named first.

MLOps already built most of the mechanics

MLOps pattern (c. 2019–2022)	**LLMOps equivalent**
Shadow deployments	Shadow-mode prompt / model rollouts
A/B testing with a randomization platform	Prompt / model A/B tests with judge + user-signal metrics
Model registry (staging → prod)	Prompt + model-version + adapter registries
Data / model lineage	Trace-level lineage: prompt hash, model version, adapter id, index
Distributional data-quality checks	Schema invariants + input drift detection
HITL on a fraction of predictions	Sampled judge eval + human review queue
CI/CD smoke tests + metrics in PRs	Prompt / model CI gates (Key Idea 3)
Monitor technical **and** business metrics	Same; business ⊇ latency, cost-per-success, judge score
Reproducibility: code + data + config	+ prompt + model version + adapter + context assembly
"ML systems fail silently" [Sculley, p. 7]	Same sentence. Same problem. Larger surface area.

Seven years later I co-wrote MLOps: A Holistic Approach (Kłeczek, Bischof & Husain, W&B 2022). The most durable idea in it: **People / Processes / Platform** *— tools alone do not ship working systems. A perfect trace infrastructure with no on-call runbook still pages the same engineer at 3 AM without a plan.*

What LLMOps changes or adds (1/2)

Dimension	MLOps world	LLMOps world
The model	You train it; you own it.	Most teams call a vendor API. Weights opaque; version bumps arrive as announcements.
Eval output	Rubrics + slice analysis + calibration + baselines; scalar metrics (AUC, F1, RMSE) where applicable.	Same discipline on text / structured outputs; rubrics extend to semantic quality; LLM-as-judge becomes central.
Versioned surface	Code + data + weights + features + thresholds.	+ prompt + tool schemas + retrieval index + context-assembly + guardrail rules.
Drift flavors	Covariate shift, label drift, concept drift, feature drift — which factor of $P(X, Y)$ moved?	Same taxonomy applies, plus drift in prompts, retrieval indices, tool outputs, user behavior.
Retry semantics	Same input $\rightarrow$ same output.	Same input $\rightarrow$ possibly *different* output. Retries can change the bug.

What LLMOps changes or adds (2/2)

Dimension	MLOps world	LLMOps world
Failure vocabulary	Class imbalance, label noise, leakage.	Hallucination, prompt injection, tool misuse, infinite loops, context rot, poor grounding.
Trace shape	Already multi-step: features → preprocessing → model(s) → postprocessing → decision logic; sometimes ensembles, cascades, shadows.	Span tree grows in width and depth: agent turns, tool calls, subagents, retrievals, guardrails. Same topology, more branches.
Regression event	New training run, threshold change, hard-rule update, ensemble reconfiguration, feature-pipeline change.	All of the above + prompt edit, vendor model bump, adapter swap, tool-schema change, retrieval-index rebuild, guardrail update.
Time to "retrain"	Hours to days — you own the pipeline.	Rarely done; fine-tuning (U12) is a rare gated event; most changes are not retraining at all.
Who owns the model?	Your team.	Usually the vendor. That inverts several MLOps defaults.

The pivot

One-line summary

In classic MLOps, **your model is your team's product**.
In LLMOps, **the model is usually someone else's infrastructure** that your product depends on.

Giving up ownership of the model in exchange for capability is the trade. Every discipline in the rest of this unit — tracing, CI, drift detection, alerting — is partly a response to that loss of control.

You can't introspect weights, you can't diff model versions, you can't schedule a retrain when things regress. What you *can* do:

observe, gate, and fall back

plus evaluate continuously — because semantic correctness cannot be summarized in a scalar.

Outline

Debugging is at least 60 years old

Most of what you do in this unit is classical technique applied to a new trace shape.

Three paradigms worth naming before the mechanics

1. **Disciplined empiricism** — the mindset. (Agans)
2. **Bisection: search for what changed** — the method. (Zeller, Torvalds)
3. **Observability over debugging** — the paradigm shift. (Majors)

Then: **three classical assumptions that LLM systems break.**

Paradigm 1 — Disciplined empiricism (Agans)

Debugging: The 9 Indispensable Rules (Agans, 2002).
Short book; reads in an evening. Four rules come up most often:

- **Understand the system.** Know the agent graph, the prompt, the retrieval index, the tool schemas *before* you poke. Bugs reported in ignorance of architecture waste hours.
- **Change one thing at a time.** Hardest rule to follow — and harder for LLMs because retries can change the output. Edit prompt + ship model upgrade in same PR, bug goes away, you have learned **nothing**.
- **Quit thinking and look.** Evidence beats theory. The trace contains the answer; most debugging time is wasted on hypotheses instead of reading spans.
- **Keep an audit trail.** Every failing trace, every fix, every decision — written down. Proto-version of the blameless postmortem + regression-test ratchet (Key Idea 3).

Paradigm 2 — Bisection: the search for what changed

Most production bugs answer to one question: **what changed since it worked?**

- **1999** — Andreas Zeller formalizes *delta debugging* (`ddmin`): binary-search an input for the minimal failing case.
- **2005** — Linus adds `git bisect`: binary-search a commit history. Differential debugging becomes a one-line UX.
- **2010s** — CI systems generalize to automatic regression attribution.
- **Today** — the surface you can bisect over is much larger, because your versioned artifacts are no longer just commits.

What transfers and what is new

`git bisect` works directly across any versioned artifact: prompts, model snapshots, adapter checkpoints, tool schemas, retrieval indices.

Two less-obvious moves:

- `ddmin` on the failing *transcript* to find a minimum reproducer.
- Bisect the *span tree* of an agent run to find the first step that diverged from the working trajectory.

Paradigm 3 — Observability over debugging (Majors)

Charity Majors, *Observability Engineering* (O'Reilly, 2022), draws a sharp line:

- **Traditional debugging / monitoring:** you know the bug exists; you go find it. Pre-wired dashboards for imagined failures. Monitoring answers **known-unknowns**.
- **Modern observability:** you cannot predict the questions you will need to ask. You emit high-cardinality, high-dimensional events so questions you didn't pre-wire are still answerable *after the fact*. Observability answers **unknown-unknowns**.

From Majors's LLM writing (Honeycomb, 2023+):

> LLMs are "a weird new kind of storage engine," and debugging them is "a classic high-cardinality, high-dimensionality problem."
>
> *LLMs demand observability-driven development.*

Production is the only real test environment for a probabilistic system. An observable production is the only debugger you get.

Three classical assumptions that break

Assumption	Classical world	LLM world + counter-move
Determinism	Same input → same output.	A **Heisenbug** — bug whose behavior changes the moment you try to observe it — is the norm. Same input → possibly different output; instrumenting can change behavior. *Counter:* snapshot the trace, replay *N* times with seed / temperature / cache controls fixed, classify the *distribution*.
Singular root cause	One bug, one fix.	Production failures are usually **multiple soft regressions aligning** (Reason's Swiss cheese). A prompt edit + a vendor bump + an unusual input cohort; none a smoking gun. *Counter:* multiple independent gates (schema, judge, red-line, drift). Look for patterns, not single causes.
Local fixes are durable	Patch the function, stays patched.	Fixes regress as the vendor updates the base model, inputs drift, or an upstream change renames a tool schema — **CACE**. Abstraction boundaries are mostly fictional in ML systems. *Counter:* every fix ships with a test in the golden set. Proof is a ratchet, not a patch.

A CACE warning before moving on

Changing Anything Changes Everything.

The debt line Sculley et al. drew under every ML system in 2015 — **louder** in LLM systems than it was in classical ML.

- A prompt edit silently changes every downstream parser.
- A base-model bump silently changes every agent that depended on that model's quirks.
- A retrieval-index rebuild silently changes what grounding each agent receives.
- A tool-schema rename silently breaks every agent that referenced the old name.

The structural defense

Test data invariants. Monitor prediction quality.
Treat **every change** as a release event.

Outline

The vendor landscape — one shared premise

The LLMOps platforms in market today — LangSmith, Braintrust, Logfire, Langfuse, Arize / Phoenix, W&B Weave, Helicone — disagree about architecture and workflow shape. Underneath the disagreement, they share one premise:

Healthy execution does not imply acceptable behavior.

Or in the form that is hard to forget once you read it:

"is it up?" $\rightarrow$ **"is it good?"**
(Braintrust, *The Three Pillars of AI Observability*, 2025)

The disagreement is about what "good" means in operational terms. **Three ideas organize it.**

Idea 1 — The triad changes

Classical observability: **metrics, logs, traces.**

AI-era equivalent, per the Braintrust post: **traces, evals, annotation.**

- **Traces:** AI spans are *orders of magnitude larger* than classical spans (tens of KB vs hundreds of bytes on average; GB-class single traces are routine).
- **Evals:** evaluation becomes a continuous production signal, not a pre-ship milestone.
- **Annotation:** human annotation of traces feeds evaluation datasets that are continuously reconciled with real traffic — not commissioned once as a "golden set."

The contested premise underneath

Evaluation is an observability signal, not a separate discipline.

Idea 2 — Evaluation as sibling, not sub-category (Majors)

Honeycomb's Charity Majors holds the opposite line, most directly in *Observability in the Age of AI*.

Observability answers	**Evaluation answers**
what happened in production.	*how good was the output.*

Same trace can feed both. The tools and the mental models stay different.

Specific argument: LLM debugging is a **high-cardinality, high-dimensional problem** — the problem class that observability (as distinct from monitoring) was invented to solve.

Folding evaluation into observability risks losing the part of observability that handles unknown-unknowns.

Idea 3 — The full-stack argument (Logfire)

Logfire's vendor-comparison pages push a structural claim beyond tool features: **AI-only observability tools are structurally incomplete.**

Production LLM failures are rarely confined to the LLM call. Three failure sites; only one is the model:

1. **What triggered the call** — upstream routing, auth, a stale cache.
2. **What the model accessed** — tool returned stale record, DB query silently failed, RAG index dropped a document.
3. **What was done with the response** — parser choked, side-effect didn't fire, webhook timed out.

The one-liner

When your AI agent misbehaves, was it the model's reasoning or the data it received? *Only full-stack observability tells you.*

Design consequences: *evals written as code* (`pydantic-evals` + `pytest`), not UI; and trace data *queryable by agents* via SQL + MCP, not just by humans through a dashboard. Majors and Logfire converge on the full-stack claim from different starting points.

Consequences for the rest of this unit

In production, these are the same bug until a trace proves otherwise:

"the agent is wrong" *"the database returned stale data"*
"the tool schema changed last Tuesday"

The operational distinction to carry forward

Eval scores tell you *that* quality dropped.
A trace that spans the LLM call, the tools that feed it, and the surrounding app code tells you *where and why*.

Most mature stacks combine both — two tools stitched via OpenTelemetry, or one platform that commits to both.

Course design choice

The rest of this unit treats **evals as a first-class signal attached to traces**, not a replacement for them.

Outline

From log lines to structured traces

A log line says *"something happened."*
A trace says *"this specific request went through these specific steps, took this long, cost this much, and produced this output."*

Every AI system you built in Units 5–12 has a natural trace shape:

- **U5 tool call:** {`tool_name, arguments, result, latency, status`}
- **U9 agent turn:** {`thought, action, observation, tool_spans[]`}
- **U10 multi-agent run:** parent span per agent, child spans per delegation, explicit handoff edges
- **U11 batch record:** {`input_id, operator_spans[], final_output, validation_result`}
- **U12 inference:** {`model, adapter_id, prompt_hash, tokens_in, tokens_out, cached`}

The common shape: **OpenTelemetry**. The LLM ecosystem is OTel with opinionated schemas on top — standardized as the **OpenTelemetry GenAI semantic conventions** (GenAI client spans, agent spans, token metrics).

What belongs in a trace

Field	Why it matters
`trace_id`, `span_id`, `parent_span_id`	Reconstruct multi-step work; required for U10 attribution.
`timestamp`, `duration_ms`	Latency analysis; SLO math.
`model`, `model_version`	Drift detection when the vendor bumps the model.
`prompt_hash` (not full prompt)	Detect prompt drift without storing PII unnecessarily.
`tokens_in`, `tokens_out`, `cost_usd`	Budget alerts; cost regression detection.
`tool_name`, `tool_args`, `tool_result`	For tool-call spans (U5).
`output_schema_valid: bool`	Cheapest, strongest signal from U11.
`eval_scores: dict`	Offline judge / rubric scores, attached when they exist.
`user_id` (hashed), `session_id`	Slice-and-dice for error analysis.
`status: ok | error | degraded`	Explicit status; not inferred from HTTP codes.

MCP tracing — one trace, many spans

MCP adds a tool-call boundary. `traceparent` header propagates `trace_id` across the boundary, so the server opens a child span under the host's call.

```
trace_id = T
→ span A · llm.tool_call           (host)
    → span B · mcp.handler         (server, parent_span_id = A)
        → span C · db.query        (server, parent_span_id = B)
        → span D · model.generate  (server, parent_span_id = B)
```

Debugging starts at the failing span and walks upward: if D failed, was the bug in D, in B that called D with bad arguments, or in A that picked the wrong tool in the first place?

That question is the first fork of any MCP-related incident — and it is only askable if `trace_id` survived the process boundary.

Traces as the eval input (closes the loop to U3)

The Unit 3 failure-mode taxonomy was built *from traces*. In production, the pattern is identical — you just **sample** instead of reviewing all of them, and the taxonomy grows with each weekly review.

The observability stack's job is to make traces:

- **Queryable** — "show me agent runs where `tool_calls` ¿ 20."
- **Linkable** — from an alert or ticket straight to the trace.
- **Replayable** — rerun *this exact trace* against a new prompt or model.
  This is the foundation of regression CI (Key Idea 3).

P^3 lens — Proof

Traces are your proof instrument at run time. Without them, "the system works" is a statement of faith. With them, every request is a replayable test case.

Outline

Most outages point at the wrong thing

The paged engineer sees **the symptom**. The trap is to fix the symptom. In AI systems the symptom is usually far from the cause:

Symptom	Typical root cause
p95 latency doubled	Input-length distribution shifted (longer contexts, new segment)
Schema-valid rate dropped	Vendor rolled a new model variant; constrained decoding regressed
User complaints about tone	Fine-tuned adapter over-fit one persona; drift in real audience
Cost up 30%	Prompt grew a "few-shot" section; cache hit rate dropped
"Agent got dumber"	Tool output format changed; agent retries, falls back
"Sometimes gives the wrong answer"	A slice (e.g. Spanish queries) regressed; aggregate metric masks it

The habit: from every alert, drill through the trace tree to the **lowest span whose behavior changed**. That is usually the cause.

Compounding errors require per-span invariants

From Unit 11:

$$0.95^5 = 0.774$$

Nearly 1 in 4 records has at least one error in a 5-step pipeline.

Corollary for observability: a *system-level* failure alert tells you almost nothing about *which step* is broken.

Fix: per-span invariants that fail loud

- A validate-after-extract span that fails with `status=error, reason=missing_required_field` pinpoints the bad extractor in one click.
- A multi-agent orchestrator with a failed child span and `status=ok` on the parent is a bug *in the harness*, not the model.

The debug-from-a-trace workflow

A reproducible loop when a production trace looks broken:

1. **Capture.** Pull the full trace (all spans, all attributes, all inputs).
2. **Replay.** Feed the same inputs to the same prompt / model / tool chain in a local / staging harness. Is the failure deterministic?
3. **Bisect.** If non-deterministic, rerun N times and classify the failure. If deterministic, binary-search the change log (prompt diffs, model version, tool schema).
4. **Minimize.** Shrink the trace to the smallest failing slice — this is the new *golden-dataset entry* for tomorrow's CI.
5. **Fix and gate.** Ship the fix *with* the new test in the same PR.

The ratchet

Every production bug that makes it to a fix also makes it to a regression test.

Outline

Golden datasets become tests

From Unit 3 you have a golden dataset, a rubric, invariants, an LLM-as-judge.
In CI those become:

- **Unit-style tests** on deterministic invariants (schema valid, required fields, no forbidden tokens).
- **Integration-style tests** on small sampled slices of the golden set, scored by rubric / judge with a pass/fail threshold.
- **Regression tests** — replay prior production traces against the new prompt / model, check outputs haven't degraded.

Mental model: `pytest` in seconds, schema invariants in minutes, judge evals in ∼10 min on a slice, full golden-set evaluation nightly.

Everything that runs pre-merge is an eval. What changes is the cost and coverage.

What to gate on a prompt change

A prompt edit is a code change. Gate it on:

Gate	Blocks merge on
Schema invariants on $N = 100$ replayed traces	any schema failure
Regression slice (judge-scored)	score drops $> X$ points vs. main
Red-line checks (U3)	any red-line violation
Cost delta	tokens / request up more than Y%
Latency delta	p95 up more than Z ms

Store prompts as versioned files (git, or a prompt registry like Braintrust / LangSmith Hub).

"Prompt changed" without a commit and a PR number is a production incident waiting to happen.

What to gate on a model upgrade

Model upgrades — even minor version bumps from the same vendor — are the **highest-risk change you routinely make**, because the diff is opaque.

The gate is strictly larger than for a prompt change:

- Full schema-invariant replay ($N \geq 1000$ historical traces).
- **Full golden-set eval** (not a slice).
- **Sliced eval by segment** — language, domain, input length, user cohort. A minor average regression often hides catastrophic slice regressions.
- Cost and latency deltas with explicit human sign-off.
- **Shadow mode** for ≥ 48 hours before promoting to primary.

Fine-tuned adapters (U12) have their *own* drift profile: base-model updates can shift adapter behavior even if the weights are identical. Gate every base-model update behind a full U12-style eval run.

Regression testing on the same trace twice

The single most valuable test you can build:

Given a stored trace, rerun end-to-end against the candidate change and diff the outputs.

The diff can be:

- **Structural** — schema field changes, missing keys.
- **Scored** — judge delta vs. prior run.
- **Human-graded** on a sampled subset.

Every production bug you fix should add one entry to this test corpus.

Offline eval $\neq$ production safety.

Distribution mismatch: your golden set is static; traffic is not.

Emergent integration failures: the LLM call passes eval; the downstream consumer misreads the subtly changed field name.

Outline

What counts as drift?

Drift is the silent killer. Three flavors, all real, each needs a separate detector:

Drift type	Definition	Typical detector
Input drift	Distribution of inputs changed (new segment, longer queries, new language).	Embed inputs; track centroid distance, PSI on length / language.
Output drift	Output distribution changed even though inputs look stable.	Rate of specific output classes; judge-score histogram over time.
Behavior drift	Users started interacting differently (more retries, more thumbs-down, more aborts).	Implicit-feedback time series; session-length distributions.

Input drift $\Rightarrow$ the world changed.
Output drift $\Rightarrow$ the model or prompt did.
Behavior drift $\Rightarrow$ the product did.
You want all three.

Sampling production traces + continuous eval

Storing every trace is fine at small scale and ruinous at large scale. From Unit 11:

Sampling pattern

Stratified sampling by slice + aggressive keep-the-bad-ones sampling (anything with `status` $\neq$ `ok`, flagged by the judge, from a new segment).

The cheapest continuous-proof loop:

1. Sample ~1–5% of production traces (stratified).
2. Run the same judge / rubric you use offline.
3. Compute scored metrics per day, per segment.
4. Alert when the score crosses the regression threshold from CI.

This is literally **Analyze–Measure–Improve** on a 24-hour clock.

Shadow mode and canaries

Before a prompt or model goes to 100% of traffic:

- **Shadow mode:** candidate runs in parallel with production; output is logged but not served. Compare outputs per trace. *Cheap; catches regressions without user impact.*
- **Canary:** 1% $\rightarrow$ 10% $\rightarrow$ 50% $\rightarrow$ 100%, with **automatic rollback gates tied to online eval scores**, not just latency.

An LLM canary that gates only on p95 latency and HTTP 5xx is almost useless.
The failure mode that matters is silent quality regression.

Outline

SLOs for LLM systems

Traditional SRE teaches SLOs on latency and availability. Both still apply.
The LLM-specific SLOs that matter most:

SLO	Example target	Fires when
Schema-valid rate	$\geq$ 99.5% parse	Decoder / model / schema drift
Judge-score median	$\geq X$ on 24h window	Quality regression
Red-line violation rate	$= 0$ (hard)	Safety failure
Tool-call success rate	$\geq$ 99%	Tool / MCP regression
p95 tokens per request	$\leq$ budget	Prompt bloat, pathological inputs
Cost per successful request	$\leq$ budget	Retries, cache misses, silent fallbacks
Human-review queue depth	$\leq N$ pending	Confidence routing too aggressive

Every one of these should be wired to a pager with a documented runbook.

Alert on eval degradation, not model version

An alert on *"model version changed"* is noisy and unhelpful.

An alert on
"judge-score median dropped 5 points on the Spanish slice in the last 24 hours"
is actionable — it points at a specific slice and a specific direction.

The on-call runbook template (one page per alert)

1. **What the alert means** (plain English).
2. **Where to look first** (trace-filter link; dashboard link).
3. **How to triage** (bisection checklist: prompt / model / tool / input).
4. **Mitigations, in order** (rollback, canary to 0%, kill switch, escalation).
5. **What to add to CI afterward** (which trace to add to the golden set).

Cost and budget alerts are eval alerts

A cost alert is a quality alert wearing a business suit.

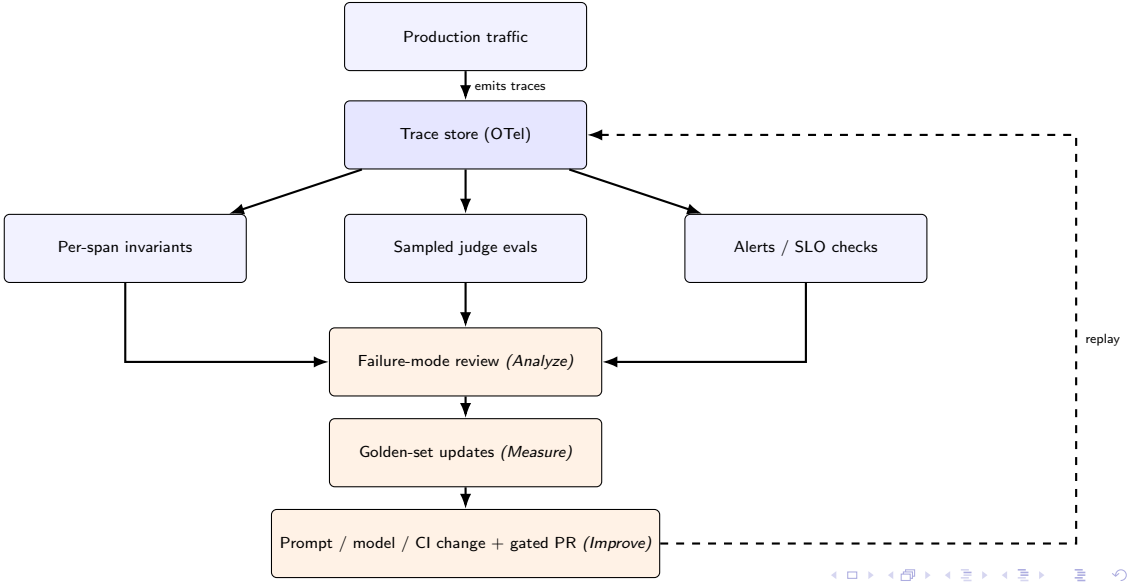
Retries, fallback chains, and silent degradations show up as **cost** first, **quality** second.

The habit

Treat a 20% unexplained cost spike the same way you'd treat a 20% drop in judge score.
It is almost always the same bug.

Outline

The observability loop



The observability loop — in one sentence

Unit 3's **Analyze–Measure–Improve** loop running forever,
with production traces as its constantly refreshed input.

The skill this unit teaches

Build the loop so it runs **without heroic on-call effort**.

The proof stays valid by *default*, not by *vigilance*.

Key takeaways

1. **Execution → Prediction → Meaning.** Observability is one discipline; the hidden state expands each era.
2. **The trace is the unit of work.** Every agent turn, pipeline record, tool call emits one. Everything downstream consumes them.
3. **Debug from traces, not from logs.** The symptom is almost never the cause; drill to the lowest span that changed.
4. **CI is the eval harness.** Golden sets, invariants, red-lines, judges run on every PR. Prompt changes and model upgrades pass the same gates as code.
5. **Monitor for drift in three places.** Inputs change, outputs change, users change — all three need their own detector.
6. **Alert on quality, not just latency.** Schema validity, judge score, red-line rate, cost per success are the SLOs that catch silent regressions.
7. **Every incident becomes a test.** Shrink the failing trace, add it to the golden set, gate future PRs on it. Proof is a ratchet.
8. **Evaluation is now a production concern.** The single biggest LLMOps-era addition over MLOps: evals don't stop when you ship. They run continuously on sampled traffic, at every change, and grow with every incident.

References (selected)

Background and prior art:

- Sculley et al., *Hidden Technical Debt in Machine Learning Systems*, NeurIPS 2015.
- Kłeczek, Bischof & Husain, *MLOps: A Holistic Approach*, W&B 2022.
- Huyen, *Designing Machine Learning Systems*, O'Reilly 2022.

Debugging lineage:

- Agans, *Debugging: The 9 Indispensable Rules*, AMACOM 2002.
- Zeller, *Yesterday, my program worked. Today, it does not. Why?*, ESEC/FSE 1999 (ddmin).
- Majors, Fong-Jones & Miranda, *Observability Engineering*, O'Reilly 2022.

Tools and specifications:

- OpenTelemetry + OTel GenAI semantic conventions (client spans, agent spans, metrics).
- Model Context Protocol.
- LangSmith, Braintrust, Logfire (Pydantic), Langfuse, Arize Phoenix, W&B Weave, Helicone.

Vendor positioning & LLM observability:

- Goyal, *The Three Pillars of AI Observability* (Braintrust, Nov 2025).
- Logfire comparison pages (Pydantic, 2025).
- Majors, *Observability in the Age of AI* (Honeycomb, 2023 / updated 2025).
- OpenAI, *Evaluation best practices*.